

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
PROGRAMMING INTERGRATION PROJECT

KNOWLEDGE RETRIEVAL AND QA FOR
COMPUTER SCIENCE COURSES UTILIZING
RAG

MAJOR: COMPUTER SCIENCE

THESIS COMMITTEE : COMPUTER SCIENCE

SUPERVISOR(s) : NGUYỄN HỨA PHÙNG, Ph.D.

—————o0o—————

STUDENT : NGUYỄN HỮU CẦU - 2352129

: NGUYỄN QUỐC HIỆU - 2352344

HO CHI MINH CITY, May 2024

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Problem statement	4
1.3	Thesis structure	5
2	Theoretical Background	7
2.1	Retrieval-Augmented Generation (RAG) pipeline	7
2.2	BERT-based Embeddings for Semantic Retrieval	8
2.2.1	Why BERT for Embeddings	8
2.2.2	Pooling and Normalization Strategies	9
2.2.3	Similarity Metrics	9
2.2.4	Embedding Dimension and Impact on Recall	10
2.2.5	Lightweight Alternatives: MiniLM	11
2.3	Vector Search	12
2.3.1	Vector-Based Retrieval Mechanism	13
2.3.2	Vector database	14
2.3.3	Role of Vector Search in Retrieval-Augmented Generation (RAG)	14
2.4	Reranking in Information Retrieval	15
2.4.1	Cross-Encoder vs Bi-Encoder Architectures	15
2.4.2	When Reranking is Needed	16
2.4.3	BGE Reranker	17
2.5	Large Language Models	17

3	Methods	20
3.1	Requirements Analysis	20
3.1.1	Functional Requirements	20
3.1.2	Non-Functional Requirements	21
3.1.3	Data Requirements	22
3.1.4	Constraints	22
3.2	System Design	23
3.2.1	System Architecture Overview	23
3.2.2	Component Design	24
3.2.2.1	Preprocessing Module	24
3.2.2.2	Embedding & Indexing Module	25
3.2.2.3	Query Pipeline Module	28
3.2.3	Data Flow	33
3.2.3.1	Indexing Flow	33
3.2.3.2	Query Flow	34
3.2.4	Technology Stack	34
4	Experiments	36
4.1	Answer Generation Quality	37
4.2	Retrieval Performance	38
4.3	Impact of Query Rewriting	39
4.4	System Performance	39
4.5	Qualitative Analysis	40
5	Results	41
5.1	Answer Generation Quality	41
5.2	Retrieval Performance	42
5.3	Impact of Query Rewriting	42
5.3.1	Impact by Query Type	42
5.4	System Performance	43

5.4.1	Performance Breakdown by Query Type	43
5.5	Qualitative Analysis	44
6	Conclusion	47
6.1	Summary	47
6.2	Achievement of Objectives	47
6.3	Key Findings	49
6.4	Future Directions	50
	References	51
	Appendix A Source Code and Dataset Availability	54

List of Figures

Figure 2.1	Bi-Encoder and Cross-Encoder architectures comparison [12]	15
Figure 2.2	BGE embedder and reranker in an RAG pipeline [12] .	17
Figure 2.3	The evolutionary tree of modern LLMs traces the development of language models in recent years [13] . .	18
Figure 3.1	Complete system architecture showing three main pipelines : Preprocessing, Embedding & Index, and Query Pipeline → API/FE. The diagram illustrates an abbreviated data flow from document ingestion through indexing to query processing and response generation.	23
Figure 3.2	Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.	24
Figure 3.3	Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS in- dex with metadata.	26
Figure 3.4	Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BGE Reranker, and response generation.	29

List of Tables

Table 4.1	Dataset Statistics	36
Table 5.1	Answer Generation Quality on CSE Course Materials	41
Table 5.2	Retrieval Performance Metrics	42
Table 5.3	Impact of Query Rewriting on Answer Quality	42
Table 5.4	Query Rewriting Impact by Query Type	43
Table 5.5	System Performance Benchmarks	43
Table 5.6	Latency by Query Type	43

Abstract

This project aims to retrieve information within the scope of CSE¹ courses by utilizing a Retrieval-Augmented Generation (RAG) system to ensure accurate information retrieval. In the current landscape, while powerful Large Language Models (LLMs) are capable of performing information gathering and question answering tasks effectively, they often encounter issues such as hallucination when controlling input information or catastrophic forgetting due to context length limitations. Therefore, integrating RAG is an effective solution to ensure the accuracy and reliability of the answers. In this project, we implement a RAG system that integrates document retrieval with the LLM to generate answers by jointly leveraging the model's parametric knowledge and non-parametric evidence retrieved from relevant documents. The system is deployed and tested using original data that we collected and processed from various sources within the CSE¹ Faculty. The results show that the system is able to generate responses that adequately satisfy users' information needs, indicating that the RAG approach is suitable for our system development direction.

¹Computer Science and Engineering

Chapter 1

Introduction

In Chapter 1, the rationale for choosing the topic and the overview of the research's project are illustrated. The outline of the report is also presented.

1.1 Motivation

In recent years, natural language processing (NLP) has become increasingly prevalent in the field of artificial intelligence, particularly in applications such as web chatbots and question-answering services. The emergence of large language models (LLMs), including ChatGPT, Gemini, and open-source models such as Llama [1] and Qwen [2], has demonstrated remarkable capabilities in understanding and generating human-like text, enabling more natural and detailed responses to user queries. However, LLMs face significant challenges in ensuring factual accuracy and domain-specific knowledge. These models, trained on general corpora, may generate plausible but factually incorrect information, especially when dealing with specialized domains such as academic course materials. This limitation poses a critical barrier for deploying LLMs in educational applications where accuracy and reliability are paramount.

Given these limitations, the development of an accurate and reliable question-answering system for educational purposes requires careful consideration of the underlying technology. Therefore, in this project, we aim to build an educational question-answering system agent that extends beyond standalone LLMs.

In considering the requirements for such a system, it is important to recognize that, indeed, the need for an educational question-answering system is particularly evident in the context of course selection and academic planning. Students frequently require timely access to course-related information, including credit hours, prerequisites, and prerequisite knowledge, when selecting courses for upcoming semesters. Currently, this process demands considerable time spent manually searching through course syllabus and documentation. Additionally, students often seek concise summaries of course content and lecture materials before class sessions or examinations, rather than reviewing entire slide sets. While LLMs present a promising solution as question-answering agents for addressing these educational needs, their aforementioned limitations in ensuring factual accuracy within specialized domains render standalone LLM-based systems inadequate for this critical application.

To address the limitations of standalone LLMs in specialized domains, retrieval-augmented generation (RAG) [3] has emerged as an effective approach that combines the generative capabilities of LLMs with external knowledge bases. RAG systems retrieve relevant information from a knowledge base and augment the LLM’s context, enabling more accurate and grounded responses. This technique has been widely adopted in various NLP applications, demonstrating significant improvements in factuality and domain-specific performance compared to standalone LLM approaches.

Motivated by these considerations, we propose a RAG-based chatbot system specifically tailored for Computer Science and Engineering (CSE) course materials. Our system leverages RAG to enable efficient retrieval of CSE course information, including credit allocations and prerequisite requirements, significantly reducing the time required for manual information retrieval compared to traditional methods. This efficiency is particularly advantageous when the underlying knowledge base remains synchronized with current semester data. The integration of LLMs further enhances the system’s effectiveness through query

rewriting techniques that improve semantic matching during the retrieval phase, while simultaneously generating coherent and user-friendly responses that facilitate comprehension.

1.2 Problem statement

The primary objective of this project is to develop an intelligent question-answering system specifically designed to assist students within the Computer Science and Engineering (CSE) faculty at HCMUT in efficiently retrieving accurate and reliable course-related information. Currently, students face significant challenges when seeking course information, including credit hours, prerequisites, course content summaries, and assessment details. The existing process requires manual navigation through extensive course details, which is time-consuming and inefficient. This project aims to address these limitations by creating an automated question-answering agent that enables rapid access to course information while maintaining high accuracy and reliability standards essential for academic applications.

The first specific objective is to enable rapid information retrieval, significantly reducing the time required for students to access course-related information compared to traditional manual search methods. The system must be capable of processing user queries and returning relevant responses within a reasonable timeframe, allowing students to quickly obtain essential information such as course prerequisites, credit allocations, and content summaries without extensive browsing through documentation.

The second critical objective is to ensure high accuracy and reliability of the information provided. Given the educational context where factual correctness is paramount, the system must generate responses that are factually accurate and grounded in the actual course materials. This requirement necessitates the integration of retrieval-augmented generation techniques that can retrieve relevant information from authoritative sources and minimize the risk of hallucination or

incorrect information generation that standalone language models may produce. The third objective encompasses addressing specific information retrieval needs within the CSE domain. The system must effectively handle queries related to course selection assistance, including providing details about credit hours, prerequisite courses, and required background knowledge. Additionally, it should support academic planning by offering concise summaries of course content and lecture materials, enabling students to quickly review key concepts before class sessions or examinations without needing to review entire slide sets or extensive documentation.

The fourth objective focuses on developing a robust and maintainable system architecture that can accommodate future expansion and enhancements. The system should be designed with modularity in mind, allowing individual components to be updated, replaced, or scaled independently. This architectural approach facilitates continuous improvement of the system's performance, enables the integration of new features, and ensures long-term maintainability without requiring extensive restructuring of the entire system.

1.3 Thesis structure

This Programming Integration Project consists of six chapters in total. Chapter 1 (Introduction), which includes the motivation, problem statement, and this thesis structure overview, has been presented above. The remaining five chapters are organized as follows:

- **Chapter 2 (Theoretical Background)** provides foundational knowledge essential for understanding the RAG system, including Retrieval-Augmented Generation (RAG) architecture, BERT-based embeddings, vector similarity search using FAISS, cross-encoder reranking with BGE Reranker, and Large Language Models (LLMs) architecture and capabilities.
- **Chapter 3 (Methods)** describes the methodology for building the RAG

system, including functional and non-functional requirements analysis, system architecture design with three main pipelines (Preprocessing, Embedding & Index, Query Pipeline), detailed component specifications (document processing, embedding generation, indexing, retrieval, reranking, and LLM integration), and technology stack selection.

- **Chapter 4 (Experiments)** presents the experimental evaluation framework, including dataset composition (32 courses, 183 documents, 15,013 text chunks), two baseline configurations (RAG Baseline with FAISS + Reranker, and RAG + Query Rewriter), evaluation metrics for answer quality (Query-Answer Similarity, Answer Faithfulness), retrieval performance (Query-Chunk Similarity, Top Chunk Score), system performance (latency, throughput), and qualitative analysis methodology.
- **Chapter 5 (Results)** visualizes and analyzes experimental results comparing different RAG pipeline configurations, including answer generation quality metrics, retrieval performance improvements, impact of query rewriting by query type, system performance benchmarks, and qualitative case studies demonstrating when query rewriting is most effective.
- **Chapter 6 (Conclusion)** summarizes the project achievements, evaluates the extent to which each objective was reached, presents key findings from the evaluation, and outlines future directions for query processing improvements and system infrastructure enhancements.

Chapter 2

Theoretical Background

In Chapter 2, it presents about fundamental knowledge in this project.

2.1 Retrieval-Augmented Generation (RAG) pipeline

RAG decouples knowledge lookup from text generation by chaining two stages: (1) **retrieve** relevant passages from a knowledge source and (2) **generate** an answer conditioned on both the query and the retrieved context [3]. The typical flow is: user query $\rightarrow$ retriever selects top- k chunks $\rightarrow$ generator produces a grounded response.

- **Retriever role.** The retriever encodes the query and documents, scores similarity, and returns the most relevant contexts. Its objective is high recall under latency and cost constraints so that the generator receives useful evidence.
- **Generator role.** The generator (e.g., an LLM) consumes the query plus retrieved contexts and crafts the final answer. It should stay grounded in the provided evidence, cite or refer to the supporting snippets, and avoid hallucinations.

Feedback and iteration. In practice, systems may refine results via iterative retrieval (e.g., re-querying based on the first draft), reranking retrieved pas-

sages, or answer verification to detect low-confidence outputs and trigger a new retrieve–then–read round. This feedback loop helps improve factuality and coverage, especially for multi-hop or under-specified questions.

2.2 BERT-based Embeddings for Semantic Retrieval

BERT (Bidirectional Encoder Representations from Transformers) [4] has become a foundational model for generating dense embeddings in information retrieval systems. Unlike traditional word-level embeddings, BERT produces contextualized representations that capture bidirectional context, making it particularly suitable for semantic similarity tasks.

2.2.1 Why BERT for Embeddings

BERT is chosen for embedding generation due to several key advantages:

- **Bidirectional context:** Unlike unidirectional language models, BERT processes text in both directions simultaneously, enabling richer semantic understanding by considering both preceding and following context for each token.
- **Pre-trained knowledge:** BERT is pre-trained on large corpora using masked language modeling and next sentence prediction objectives, encoding substantial linguistic and world knowledge that transfers well to downstream tasks.
- **Task adaptability:** Through fine-tuning or feature extraction, BERT embeddings can be adapted for various retrieval tasks without requiring task-specific architectural changes.

However, vanilla BERT is not optimized for producing sentence-level embeddings suitable for similarity search. Sentence-BERT (SBERT) [5] addresses

this limitation by fine-tuning BERT with siamese and triplet network structures on sentence-pair datasets, enabling efficient semantic similarity computation.

2.2.2 Pooling and Normalization Strategies

Converting BERT's token-level outputs to fixed-size sentence embeddings requires pooling strategies:

- **Mean pooling:** Computes the average of all token embeddings (excluding special tokens). This approach captures information from all tokens and is empirically shown to outperform CLS token pooling for semantic similarity tasks [5].
- **CLS token pooling:** Uses the [CLS] token's embedding as the sentence representation. While designed for classification tasks, it often underperforms mean pooling for retrieval due to limited semantic information.
- **Max pooling:** Takes the maximum value across tokens for each dimension. Less commonly used for semantic tasks as it may emphasize outlier features.

Normalization is crucial for embedding quality. L2 normalization (unit vector scaling) is commonly applied to embeddings before similarity computation. This normalization:

- Reduces the impact of embedding magnitude on similarity scores
- Improves training stability and convergence
- Makes cosine similarity equivalent to dot product for normalized vectors

2.2.3 Similarity Metrics

The choice of similarity metric directly impacts retrieval performance:

- **Cosine similarity:** Measures the cosine of the angle between two vectors: $\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$. For L2-normalized embeddings, this reduces to dot product. Cosine similarity is scale-invariant and widely used for semantic retrieval.
- **Inner product (dot product):** Computes $\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$. When embeddings are normalized, inner product equals cosine similarity. Inner product is computationally efficient and directly optimizable in vector search libraries like FAISS.
- **Euclidean distance:** Measures L_2 distance: $\|\mathbf{u} - \mathbf{v}\|_2$. Less common for semantic similarity as it is sensitive to vector magnitude.

For normalized embeddings, cosine similarity and inner product are equivalent and both are commonly used in production systems.

2.2.4 Embedding Dimension and Impact on Recall

The embedding dimension (typically 768 for BERT-base, 1024 for BERT-large) affects both retrieval quality and computational efficiency:

- **Higher dimensions** can capture more nuanced semantic information, potentially improving recall for complex queries. However, they increase storage requirements and search latency.
- **Lower dimensions** (achieved via dimensionality reduction) reduce computational cost but may lose semantic expressiveness, potentially decreasing recall.
- **Recall trade-offs:** In approximate nearest neighbor search (e.g., FAISS), higher-dimensional embeddings require more memory and may need larger index parameters (e.g., more centroids in IVF) to maintain recall. The choice of dimension balances between semantic expressiveness and search efficiency.

Empirical studies show that BERT-base embeddings (768 dimensions) with mean pooling and L2 normalization provide a good balance between quality and efficiency for most retrieval tasks, achieving strong recall while remaining computationally tractable for large-scale vector databases.

2.2.5 Lightweight Alternatives: MiniLM

While BERT-base and BERT-large provide high-quality embeddings, their computational requirements (110M and 340M parameters respectively) can be prohibitive for resource-constrained deployments. MiniLM [6] addresses this limitation through knowledge distillation, compressing larger transformer models into smaller, faster variants while preserving semantic quality.

MiniLM architecture: MiniLM uses deep self-attention distillation to transfer knowledge from teacher models (e.g., BERT-base) to student models with fewer layers and hidden dimensions. The distillation process focuses on matching attention distributions and hidden states, enabling the student to capture essential semantic patterns.

all-MiniLM-L6-v2: The all-MiniLM-L6-v2 model, part of the Sentence-Transformers library, is a distilled version optimized for sentence embeddings. Key characteristics:

- **Compact size:** 22.7M parameters (compared to 110M for BERT-base), enabling faster inference and lower memory footprint.
- **Lower dimensionality:** 384-dimensional embeddings (vs. 768 for BERT-base), reducing storage requirements and search latency in vector databases.
- **Performance:** Despite being smaller, MiniLM maintains competitive semantic quality on benchmark tasks, often achieving 95–98% of BERT-base performance with 2–3x faster inference.

Trade-offs and selection criteria: The choice between BERT-base and MiniLM involves balancing:

- **Quality vs. efficiency:** BERT-base offers marginally better semantic understanding for complex queries, while MiniLM provides faster batch processing and lower resource consumption.
- **Recall impact:** For most retrieval tasks, the recall difference between 384-dimensional MiniLM and 768-dimensional BERT-base embeddings is minimal (typically <2%), making MiniLM suitable for production systems prioritizing latency and cost.
- **Scalability:** Lower-dimensional embeddings enable more efficient FAISS indexing, allowing larger candidate sets to be searched within the same memory budget.

In practice, MiniLM is often preferred for production RAG systems where inference speed and resource efficiency are critical, while maintaining acceptable retrieval quality. The 384-dimensional embeddings provide sufficient semantic expressiveness for most domain-specific retrieval tasks while enabling real-time query processing.

2.3 Vector Search

Vector search [7] constitutes a fundamental paradigm for semantic information retrieval, in which documents and queries are represented as points in a continuous vector space and relevance is determined by geometric proximity rather than lexical overlap. Unlike traditional keyword-based retrieval methods, vector search enables the retrieval of semantically related content even when there is little or no surface-level term matching, making it particularly suitable for modern language model-based systems.

From a conceptual perspective, vector search should not be viewed merely as a nearest neighbor algorithm, but rather as a retrieval abstraction that integrates representation learning, similarity metrics, indexing strategies, and system-level

trade-offs. This abstraction forms the backbone of dense retrieval systems and plays a critical role in Retrieval-Augmented Generation (RAG) pipelines [3].

2.3.1 Vector-Based Retrieval Mechanism

In vector search, each document or text segment is mapped to a dense vector

$$\mathbf{v}_i \in \mathbb{R}^d$$

using an embedding model [5], while a user query is encoded into a query vector

$$\mathbf{q} \in \mathbb{R}^d$$

within the same vector space.

The objective of the retrieval process is to identify the set of document vectors that are most similar to the query vector. Formally, this can be expressed as:

$$\text{top-k}(\mathbf{q}) = \arg \max_{\mathbf{v}_i} s(\mathbf{q}, \mathbf{v}_i),$$

where $s()$ denotes a similarity function.

Common similarity measures include cosine similarity, inner product, and Euclidean distance. In practice, cosine similarity is widely adopted for semantic retrieval tasks due to its robustness to vector magnitude. When vectors are L2-normalized, cosine similarity becomes equivalent to the inner product:

$$\cos(\mathbf{q}, \mathbf{v}) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\| \|\mathbf{v}\|}.$$

Because embeddings generated by modern language models often reside in high-dimensional spaces, exhaustive linear search can become computationally expensive. As a result, vector search systems employ various indexing strategies and search algorithms, including exact search and approximate nearest neighbor methods, to balance retrieval accuracy and efficiency.

2.3.2 Vector database

With the growing adoption of vector-based retrieval, a variety of vector databases have been developed to support large-scale storage and search over embedding vectors. These systems typically integrate vector search capabilities with persistent storage, metadata management, conditional filtering, and distributed deployment, targeting production environments with dynamic and large-scale data. Representative vector database systems include *Qdrant* [8], which emphasizes approximate nearest neighbor search combined with payload-based filtering, *Milvus* [9], which provides a distributed architecture for large-scale vector data management, and *Weaviate* [10], which integrates vector search with schema-aware data modeling and API-based access. These systems illustrate a service-oriented approach in which vector search is tightly coupled with database-level functionalities.

In parallel, vector search libraries focus primarily on efficient similarity computation and indexing, without providing full database functionalities. Rather than serving as standalone data management systems, these libraries act as core computational components that can be embedded into broader retrieval pipelines.

FAISS is a representative example of vector search libraries, designed specifically for efficient similarity search over dense vectors in high-dimensional spaces. In the context of RAG systems, FAISS is often referenced as a canonical implementation of library-based vector search, while other vector database systems exemplify service-oriented approaches tailored for large-scale production use .

2.3.3 Role of Vector Search in Retrieval-Augmented Generation (RAG)

Within a Retrieval-Augmented Generation framework [3], vector search typically functions as the first-stage retriever. Its primary role is to identify a set of candidate text segments that exhibit high semantic relevance to a given query.

The effectiveness of this retrieval stage has a direct impact on subsequent components, such as reranking mechanisms and language model–based answer generation. A vector search system with high recall helps ensure that relevant contextual information is made available to downstream models, thereby supporting more accurate and contextually grounded responses.

For this reason, vector search is widely regarded as a foundational component in RAG architectures, serving as the critical link between semantic representation learning and generative language modeling.

2.4 Reranking in Information Retrieval

Reranking is a two-stage retrieval strategy that refines initial retrieval results by applying a more sophisticated scoring model to a small candidate set [11]. In the first stage, a fast retriever (typically a bi-encoder) retrieves a large set of candidates (e.g., top- k with $k = 100$). In the second stage, a reranker (typically a cross-encoder) re-scores these candidates to produce the final ranked list.

2.4.1 Cross-Encoder vs Bi-Encoder Architectures

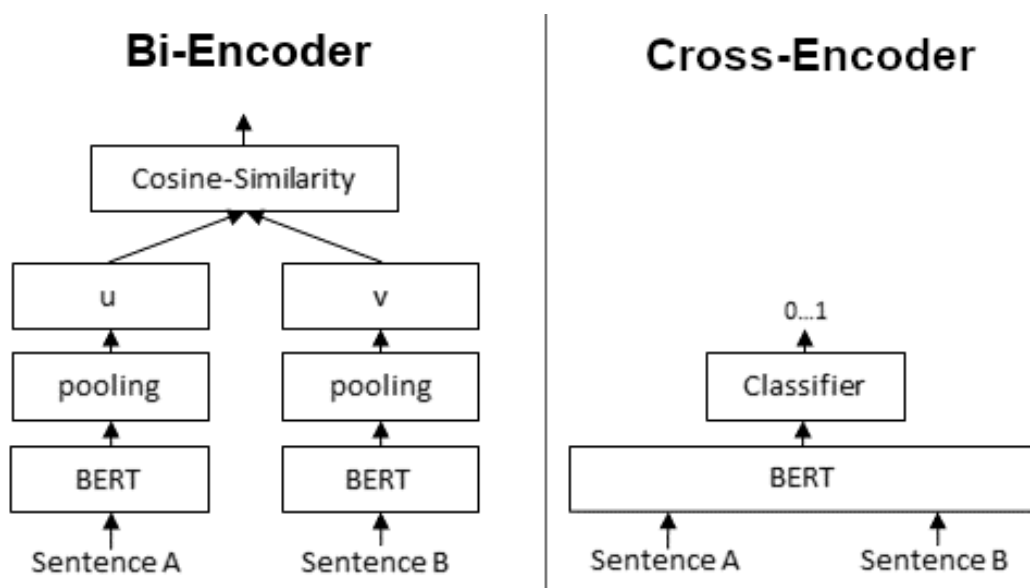


Figure 2.1: Bi-Encoder and Cross-Encoder architectures comparison [12]

The choice between cross-encoder and bi-encoder architectures represents a fundamental trade-off between accuracy and efficiency:

- **Bi-encoder** (used in initial retrieval): Query and document are encoded separately into dense vectors. Similarity is computed via dot product or cosine similarity. This enables efficient approximate nearest neighbor search (e.g., using FAISS, Qdrant) with sub-linear time complexity, making it suitable for large-scale retrieval. However, bi-encoders cannot model fine-grained query–document interactions.
- **Cross-encoder** (used in reranking): Query and document are concatenated and fed into a single transformer model, enabling joint attention over both inputs. This allows the model to capture complex semantic interactions and fine-grained relevance signals, typically achieving higher accuracy than bi-encoders. However, cross-encoders require computing scores for each query–document pair independently, making them computationally expensive for large candidate sets.

The two-stage approach leverages the strengths of both: bi-encoders provide fast candidate retrieval, while cross-encoders provide accurate final ranking on a small subset.

2.4.2 When Reranking is Needed

Reranking becomes particularly valuable when:

- The initial retriever has high recall but lower precision, requiring refinement to identify the most relevant passages.
- The downstream task (e.g., LLM generation) benefits from higher-quality context, where even small improvements in top- k relevance can significantly impact answer quality.

- Computational budget allows for the additional latency cost of reranking a small candidate set (typically 10–50 items).

2.4.3 BGE Reranker

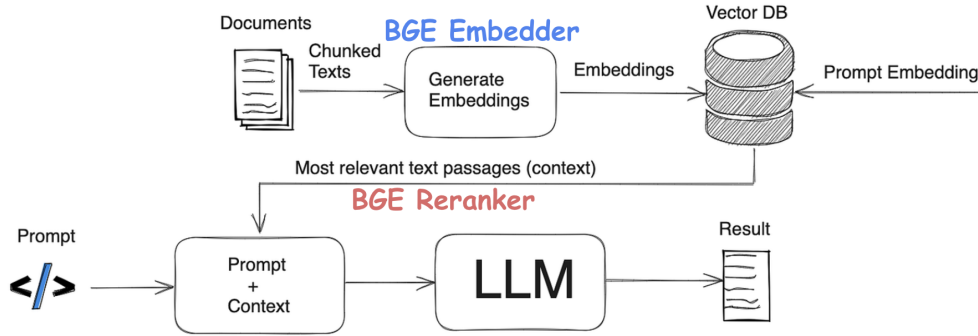


Figure 2.2: BGE embedder and reranker in an RAG pipeline [12]

The BGE (BAAI General Embedding) reranker is a cross-encoder model developed by the Beijing Academy of Artificial Intelligence (BAAI) [12]. Built on XLM-RoBERTa architecture, it is specifically designed for reranking tasks in both English and Chinese. The model employs a cross-encoder architecture where query and passage are concatenated and processed jointly, enabling fine-grained relevance scoring. BGE reranker models (base and large variants) have demonstrated state-of-the-art performance on various retrieval benchmarks, making them suitable for production RAG systems where accuracy is prioritized over minimal latency.

2.5 Large Language Models

Large Language Models that are pre-trained on large-scale corpus have demonstrated significant potential in various natural language processing tasks. Most LLMs are based on the Transformer design, which consists of encoder and decoder modules that are empowered by a self-attention mechanism. Based on their architecture structure, LLMs can be classified into three categories:

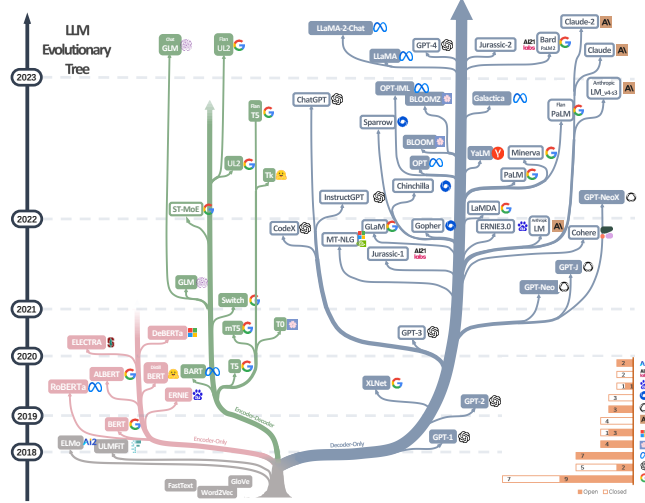


Figure 2.3: The evolutionary tree of modern LLMs traces the development of language models in recent years [13]

encoder-only LLMs, encoder-decoder LLMs, and decoder-only LLMs. Figure 2.3 summarizes several representative LLMs with different model architectures, model sizes, and open-source availabilities.

Encoder-only LLMs are a type of Large Language Models that use only the encoder to encode the sentence and understand the relationships between words. These models are typically pre-trained by predicting the mask words in an input sentence using large-scale corpus. To resolve downstream tasks, such as text classification and named entity recognition, an extra prediction head is added to encoder-only LLMs like BERT [4], RoBERTa [14], ALBERT [15], and ELECTRA [16]. These models are most effective for tasks that require understanding the entire sentence.

Encoder-decoder LLMs adopt both the encoder and decoder module. The encoder module is responsible for encoding the input sentence into a hidden space, and the decoder is used to generate the target output text. The training strategies in encoder-decoder LLMs can be more flexible. Encoder-decoder LLMs are able to directly resolve tasks that generate sentences based on some context,

such as summariaztion, translation, and question answering.

Decoder-only LLMs only adopt the decoder module to generate target output text. These models are trained to predict the next word in a sentence. They can perform downstream tasks with few examples or simple instructions without adding prediction heads or fine-tuning [17]. Many state-of-the-art LLMs follow the decoder-only architecture. Recently Llama-2 [1], Qwen [2], Alpaca¹ and Vicuna² are released as open-source decoder-only LLMs. These models are finetuned based on LLaMA [18] and achieve comparable performance with ChatGPT [19] and GPT-4

¹<https://crfm.stanford.edu/2023/03/13/alpaca.html>

²<https://lmsys.org/blog/2023-03-30-vicuna/>

Chapter 3

Methods

In Chapter 3, it presents the methodology for building the RAG system [3], including requirements analysis, system design, and detailed implementation of preprocessing, embedding, indexing, and LLM-powered query optimization.

3.1 Requirements Analysis

This section analyzes the functional and non-functional requirements for developing a RAG-based questioning-answering system [3] tailored for Computer Science course materials at HCMUT.

3.1.1 Functional Requirements

The system is designed to collect course materials from multiple sources, including the HCMUT LMS [20] and Coursewave [21], and will support various document formats such as PDF and PPTX. It will utilize OCR technology, specifically PaddleOCR [22], [23], to extract text from scanned documents and parse syllabus documents to capture essential metadata. In order to facilitate semantic searches, the system will chunk text documents into overlapping segments, generate vector embeddings using a pre-trained model sentence-transformers/all-MiniLM-L6-v2 [5], [6], and build FAISS vector indices [7] for efficient similarity searching. Moreover, the system will maintain metadata mappings, linking

vector IDs to original document sources and other relevant information, allowing for both per-course and cross-course retrieval.

For query processing, the system will accept natural language questions from users and may employ a large language model (LLM) [13] to rephrase queries for enhanced retrieval quality. It will retrieve the top- k document chunks based on semantic similarity and use a cross-encoder reranker [11], [12] to improve the relevance of these chunks, providing source citations for each response.

In terms of answer generation, the system will leverage a large language model to formulate answers based on the context obtained from the retrieved chunks. It will support multiple LLM providers, such as Google Gemini, Ollama, and Qwen [2], allowing for configurable models. The generated answers will include proper citation of sources and will handle cases where no relevant context is found by providing appropriate feedback.

The user interface will feature a web-based chat interface for user interactions, displaying conversation history and enabling follow-up questions. It will also show source citations for each answer and provide an API endpoint to allow programmatic access.

3.1.2 Non-Functional Requirements

The system is expected to efficiently retrieve relevant chunks within 500 milliseconds for queries against a corpus of over 10,000 chunks, generating answers below 30 seconds, encompassing retrieval, reranking, and answer formulation. The embedding model will produce embeddings of 384 dimensions, striking a balance between quality and efficiency, while also supporting batch processing for large document collections.

Regarding scalability, the system will facilitate indexing materials across more than 50 courses, support incremental updates of the vector index structure, and maintain a modular architecture to enable independent scaling of components.

To ensure reliability and availability, the system must gracefully handle API

errors and maintain data integrity during both indexing and retrieval operations. Health check endpoints will be provided for monitoring system performance and status.

Usability will be prioritized through an intuitive user interface that supports natural language queries without the need for specific syntax. Generated responses will be clear, concise, and adequately formatted. The codebase will follow modular design principles for easy maintenance and be containerized using Docker for consistent deployment across various environments.

3.1.3 Data Requirements

The input data will comprise course syllabi and lecture materials in PDF and PPTX formats, allowing for the extraction of chapter numbers, page indices, and hierarchical content organization. The output will include structured JSON files for parsed content, vector embeddings stored in FAISS indices [7] with the associated metadata, and generated answers formatted with source citations for API responses.

To maintain data quality, the OCR process [22] will focus on achieving sufficient accuracy for extracting readable text from scanned documents, while the chunking strategy will aim for semantic coherence within the chunks. Metadata must also be complete and accurate to ensure proper source attribution.

3.1.4 Constraints

The system will be designed to run efficiently on CPU without requiring GPU resources for inference. It will be focused specifically on Computer Science course materials at HCMUT, with no need for generalization to other domains. The knowledge base will be kept static, and real-time updates from external sources will not be supported. The availability of LLM APIs will rely on external services (like Gemini API) or local setups (such as Ollama or Qwen [2]).

3.2 System Design

This section presents the overall architecture and design of the RAG system for CSE course materials. The system is designed as a modular pipeline with distinct stages for data processing, indexing, and query answering.

3.2.1 System Architecture Overview

Figure 3.1 illustrates the complete system architecture with all three main pipelines and their interactions.

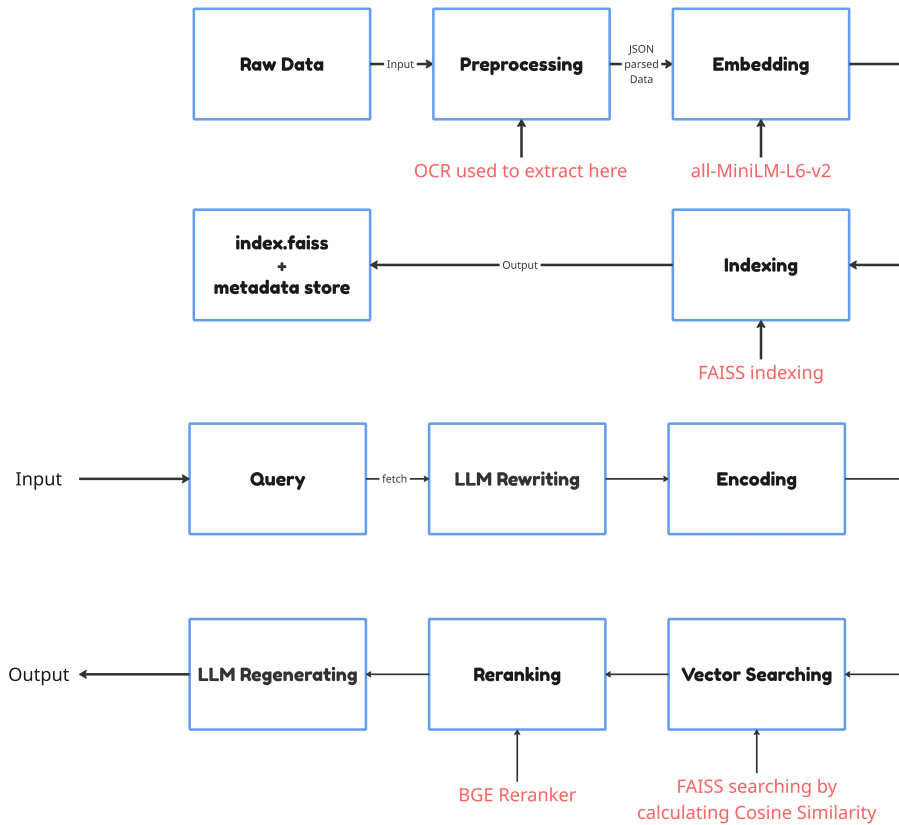


Figure 3.1: Complete system architecture showing three **main pipelines**: Pre-processing, Embedding & Index, and Query Pipeline → API/FE. The diagram illustrates an abbreviated data flow from document ingestion through indexing to query processing and response generation.

The system follows a three-stage architecture.

Preprocessing Pipeline. The pipeline processes raw documents, extracts text using OCR [22], parses structured information, and chunks text for indexing.

Embedding & Index Pipeline. The pipeline converts text chunks to vector embeddings, normalizes them, and stores them in FAISS indices [7] with metadata.

Query Pipeline. It handles user queries through API/Frontend, retrieves relevant chunks, reranks results [12], and generates answers by LLM [13].

3.2.2 Component Design

3.2.2.1 Preprocessing Module

The preprocessing module handles document ingestion and text extraction. Figure 3.2 shows the detailed preprocessing pipeline.

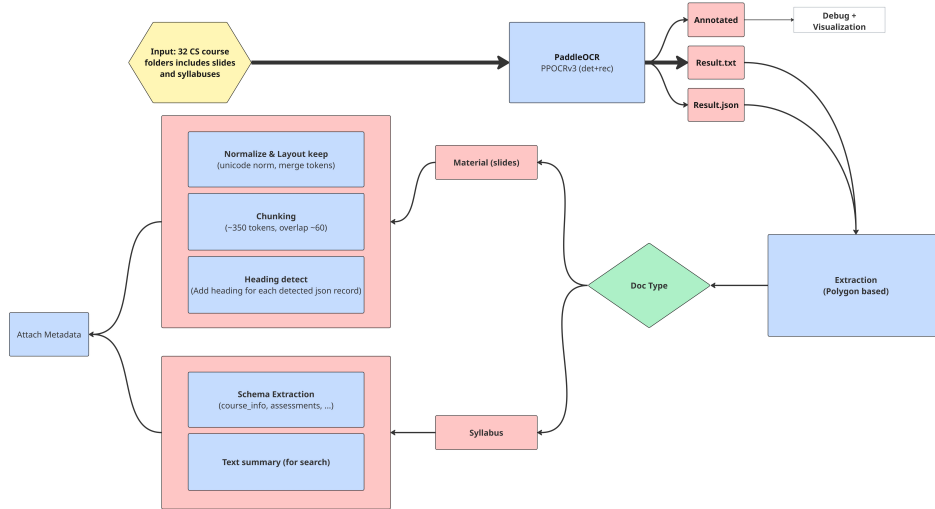


Figure 3.2: Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.

The preprocessing module consists of several key components.

Document Converter. It converts PDF/DOCX/PPTX files to page images at 220 DPI resolution for OCR processing.

OCR Engine. It utilizes PaddleOCR 3.2 [22], [23] to extract text with bounding box information and confidence scores, enabling accurate text extraction from scanned documents and images.

Syllabus Parser. It extracts structured information from syllabus documents including course metadata, grading policies, and learning objectives. **Material Parser.** It processes lecture slides to extract chapter information, page indices, and content structure.

Text Chunker. It segments documents into overlapping chunks using a sliding window approach with configurable window size and overlap, ensuring that semantic context is preserved across chunk boundaries.

3.2.2.1.1 Chunking Algorithm The chunking process uses a sliding window with token-based segmentation. Given a document with T tokens, window size S (default 512 tokens), and overlap O (default 50 tokens), the expected number of chunks is computed as:

$$\text{chunks}(T; S, O) \approx \left\lceil \frac{\max(T - S, 0)}{S - O} \right\rceil + 1$$

where $S - O$ represents the stride between consecutive windows. The implementation uses tiktoken with the cl100k_base vocabulary for accurate token counting, ensuring consistent chunk boundaries across different document types.

3.2.2.2 Embedding & Indexing Module

The embedding and indexing modules work together to create searchable vector representations. Figure 3.3 illustrates the embedding generation, normalization, and indexing process.

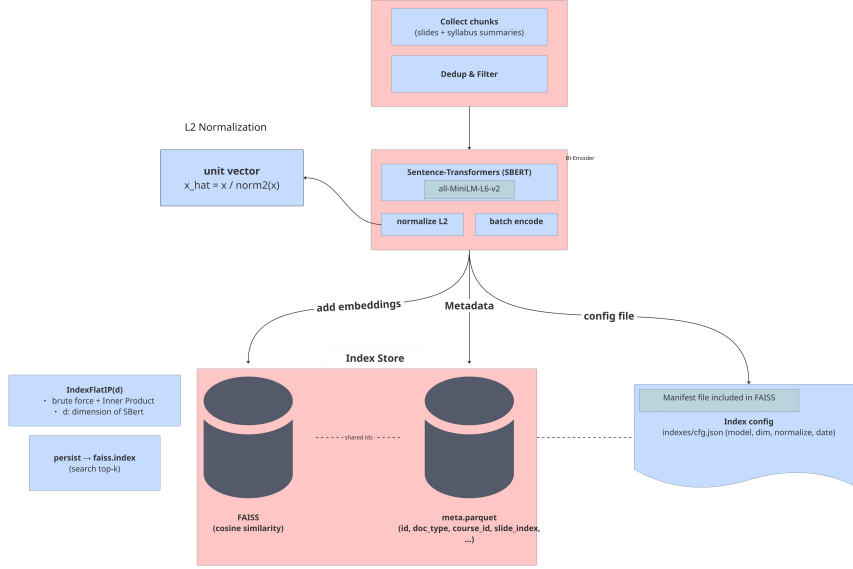


Figure 3.3: Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS index with metadata.

The embedding module converts text into vector representations through a multi-stage process.

Embedding Model. It employs sentence-transformers/all-MiniLM-L6-v2 [5], [6] to generate 384-dimensional embeddings that capture semantic meaning of text chunks.

Batch Processor. it processes chunks in batches for efficient embedding generation, optimizing computational resources.

Normalization Component. It L2-normalizes embeddings using the formula

$$\hat{x} = x / \|x\|_2$$

to create unit vectors, which enables cosine similarity computation via inner product operations. The **FAISS Index** [7] utilizes IndexFlatIP (Inner Product) for exact similarity search on normalized vectors, providing precise retrieval results. The **Metadata Store** maintains JSON mappings from vector IDs to

document metadata including course ID, page index, chapter, and source file information, enabling traceability of retrieved content. The **Index Manager** handles index creation, updates, and persistence to disk, ensuring data durability and efficient index management across multiple courses.

3.2.2.2.1 Embedding Generation Embedding generation is performed in batches for computational efficiency. The process converts text chunks to 384-dimensional vectors using the sentence-transformers/all-MiniLM-L6-v2 model [5], [6]:

Algorithm 1 Batch Embedding Generation

Input: List of text chunks $C = \{c_1, c_2, \dots, c_n\}$, batch size b , embedding model M

Output: Array of embeddings E where each $e_i \in \mathbb{R}^{384}$

- 1: Initialize empty list $E \leftarrow []$
 - 2: Extract text from chunks: $T \leftarrow \{c_i.\text{text} \mid c_i \in C\}$
 - 3: **for** $i = 0$ to $\lceil |T|/b \rceil - 1$ **do**
 - 4: $B \leftarrow T[i \cdot b : (i + 1) \cdot b]$ {Get batch of texts}
 - 5: $E_b \leftarrow M.\text{embed_batch}(B)$ {Generate embeddings for batch}
 - 6: $E \leftarrow E \cup E_b$ {Append batch embeddings}
 - 7: **end for**
 - 8: Convert E to numpy array with dtype float32
 - 9: **return** E
-

3.2.2.2.2 Indexing Algorithm The indexing process creates a FAISS index and stores normalized embeddings with metadata. The algorithm proceeds as follows:

Algorithm 2 FAISS Index Creation and Normalization

Input: Embeddings $E \in \mathbb{R}^{n \times 384}$, metadata map M , embedding dimension $d = 384$

Output: FAISS index I saved to disk, metadata file saved

```
1: Create FAISS index:  $I \leftarrow \text{IndexFlatIP}(d)$ 
2: Compute L2 norms:  $N \leftarrow \|E\|_2$  for each row
3: for each norm  $n_i \in N$  do
4:   if  $n_i = 0$  then
5:      $n_i \leftarrow 1$  {Avoid division by zero}
6:   end if
7: end for
8: Normalize embeddings:  $\hat{E} \leftarrow E/N$  {L2 normalization}
9: Convert  $\hat{E}$  to float32 format
10: Add normalized embeddings to index:  $I.\text{add}(\hat{E})$ 
11: Save index to disk:  $\text{faiss.write\_index}(I, \text{"index.faiss"})$ 
12: Save metadata:  $\text{json.dump}(M, \text{"metadata.json"})$ 
```

This normalization ensures that cosine similarity between vectors u and v can be computed as an inner product: $\text{cosine}(u, v) = u \cdot v$ when both vectors are unit length.

3.2.2.3 Query Pipeline Module

The query pipeline module handles the complete flow from user query to response generation. Figure 3.4 shows the detailed query processing architecture including API backend, reranking, and frontend interaction.

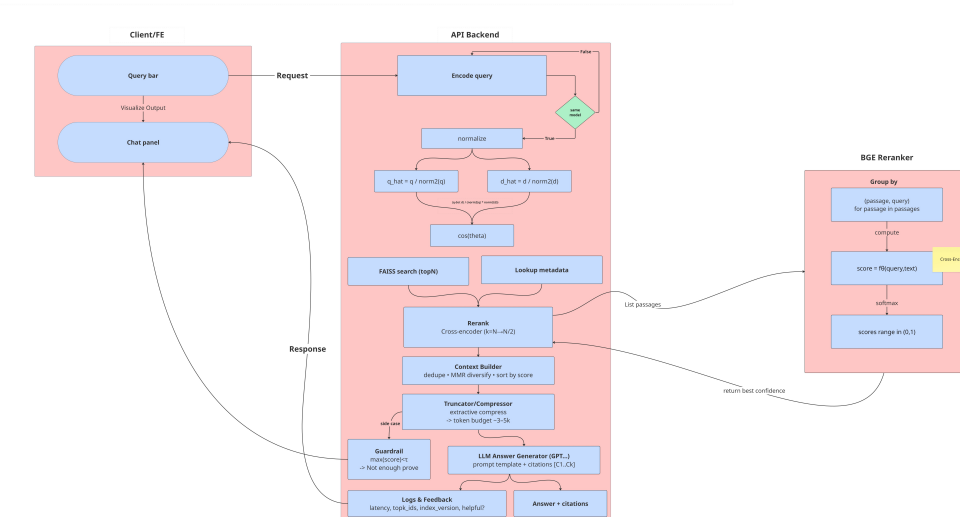


Figure 3.4: Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BGE Reranker, and response generation.

The query pipeline processes user queries and generates responses through multiple interconnected components.

Query Rewriter. It uses LLM [13], such as Qwen [2], to rewrite user queries for improved retrieval quality, enhancing the semantic matching between queries and documents.

Query Embedder. It converts queries to embeddings using the same model as document embeddings, ensuring consistency in the vector space.

Vector Searcher. It performs top- k similarity search using FAISS [7] and fetches results by IDs, enabling fast retrieval from large document collections.

Reranker. It utilizes FlagEmbedding cross-encoder (BGE Reranker) [12] to rerank retrieved chunks by relevance, improving the precision of the final results.

LLM Client. It abstracts interactions with different LLM providers including

Gemini, Ollama, and Qwen [2], providing a unified interface for answer generation [13]. Then the **Context Formatter** prepares retrieved chunks as context for the LLM prompt, organizing information in a format suitable for generation. Eventually, **Response Generator**. It calls the LLM with formatted prompt to generate answers, leveraging the retrieved context to produce accurate and relevant responses.

3.2.2.3.1 Vector Search Algorithm The search process normalizes the query embedding and performs top- k similarity search using FAISS:

Algorithm 3 Top- k Vector Similarity Search

Input: FAISS index I , query embedding $q \in \mathbb{R}^{384}$, number of results k , metadata map M

Output: List of results R containing $(chunk_id, score, metadata)$ tuples

```

1: Compute query norm:  $n_q \leftarrow \|q\|_2$ 
2: if  $n_q = 0$  then
3:    $n_q \leftarrow 1$  {Avoid division by zero}
4: end if
5: Normalize query:  $\hat{q} \leftarrow q/n_q$ 
6: Convert  $\hat{q}$  to float32 and reshape to  $(1, 384)$ 
7: Perform FAISS search:  $(D, Idx) \leftarrow I.search(\hat{q}, k)$ 
8: Initialize empty list:  $R \leftarrow []$ 
9: for each  $(score, idx) \in (D[0], Idx[0])$  do
10:    $chunk\_id \leftarrow M[idx][\"id\"]$ 
11:    $metadata \leftarrow M[idx]$ 
12:   Append  $(chunk\_id, score, metadata)$  to  $R$ 
13: end for
14: return  $R$ 

```

The search latency is typically 1–10 ms for thousands of chunks, enabling real-time query processing.

3.2.2.3.2 Reranking Algorithm After initial retrieval, the system applies a cross-encoder reranker [12] to refine results. The BGE reranker uses a cross-encoder architecture where the query q and passage p are concatenated and processed jointly:

$$\text{Input} = [\text{CLS}] \oplus q \oplus [\text{SEP}] \oplus p \oplus [\text{SEP}]$$

The relevance score is computed as:

$$s(q, p) = \text{Transformer}([\text{CLS}] \oplus q \oplus [\text{SEP}] \oplus p \oplus [\text{SEP}])$$

Raw scores are normalized using softmax for confidence estimation:

$$\text{confidence}_i = \frac{\exp(s_i - s_{\max})}{\sum_{j=1}^n \exp(s_j - s_{\max})}$$

where $s_{\max} = \max(s_1, s_2, \dots, s_n)$ is the maximum raw score (for numerical stability). The implementation:

Algorithm 4 Cross-Encoder Reranking with Softmax Normalization

Input: Query string q , list of retrieved chunks $C = \{c_1, c_2, \dots, c_n\}$, reranker model R , top- k value k

Output: Ranked list of top- k chunks with confidence scores

- 1: Create query–passage pairs: $P \leftarrow \{(q, c_i.\text{text}) \mid c_i \in C\}$
 - 2: Compute raw relevance scores: $S \leftarrow \{R.\text{compute_score}(p) \mid p \in P\}$
 - 3: Find maximum score: $s_{\max} \leftarrow \max(S)$
 - 4: Initialize confidence list: $Conf \leftarrow []$
 - 5: **for** each score $s_i \in S$ **do**
 - 6: $exp_score \leftarrow \exp(s_i - s_{\max})$ {Numerical stability}
 - 7: Append exp_score to $Conf$
 - 8: **end for**
 - 9: Compute normalizer: $Z \leftarrow \sum(Conf)$
 - 10: **if** $Z = 0$ **then**
 - 11: $Z \leftarrow 1.0$ {Avoid division by zero}
 - 12: **end if**
 - 13: Normalize confidences: $Conf \leftarrow \{c_i/Z \mid c_i \in Conf\}$
 - 14: Create tuples: $Ranked \leftarrow \{(c_i, Conf[i]) \mid c_i \in C\}$
 - 15: Sort by confidence (descending): $Ranked \leftarrow \text{sort}(Ranked, \text{key} = \text{confidence}, \text{reverse} = \text{true})$
 - 16: **return** First k elements of $Ranked$
-

This two-stage approach (FAISS retrieval + reranking) improves precision by allowing the cross-encoder to capture fine-grained query–passage interactions that are not captured by embedding similarity alone.

3.2.3 Data Flow

3.2.3.1 Indexing Flow

The indexing pipeline processes documents through a sequential workflow designed to transform raw documents into searchable vector representations.¹ The process begins with **Document Ingestion**, where raw documents in PDF, DOCX, or PPTX formats are placed in the directory structure `data/raw/<CourseName>`/². During the **Conversion** stage, documents are converted to page images at 220 DPI resolution, which are stored in `data/converted`/³ for subsequent OCR processing. The **OCR Processing** stage utilizes PaddleOCR [22], [23] to extract text and structure information, producing JSON files containing text content, bounding boxes, and confidence scores in `data/data`/⁴. The **Parsing** stage applies specialized parsers for syllabus and material documents to extract structured information such as course metadata, chapter organization, and page indices. During **Chunking**, text is segmented into overlapping chunks with associated metadata, preserving semantic context across boundaries. The **Embedding** stage converts text chunks to vector embeddings using the sentence-transformer model, generating 384-dimensional representations. Finally, the **Indexing** stage stores embeddings in FAISS indices along with metadata mappings in `data/indices`/⁵, enabling efficient similarity search across the document corpus.

¹The `/data` folder serves as the common storage location for all preprocessing data.

²The `/data/raw` folder contains the original input documents organized by course name, where each course has its own subdirectory.

³The `/data/converted` folder (where `converted` is nested within `data`) contains files converted from PDF to PNG format.

⁴The `/data/data` folder (where `data` is nested within the parent `data` folder) contains data extracted from converted files using OCR [22], [23].

⁵The `/data/indices` folder contains metadata and `index.faiss` files for each course.

3.2.3.2 Query Flow

The query pipeline processes user questions through a multi-stage retrieval and generation workflow. The process initiates with **Query Input**, where users submit natural language questions via the web UI or REST API. An optional **Query Rewriting** stage employs an LLM, such as Qwen [2], to rewrite the query for improved retrieval quality, enhancing semantic matching with document content. During **Query Embedding**, the query is converted to a vector embedding using the same model employed for document embeddings, ensuring consistency in the vector space. The **Vector Search** stage utilizes FAISS [7] to retrieve top- k similar chunks (with default $k = 10$), performing efficient similarity search across the indexed document corpus. The **Reranking** stage applies a cross-encoder reranker [11], [12] to score and rerank retrieved chunks, improving precision by considering query-document interactions. During **Context Preparation**, the top reranked chunks are formatted as context for the language model, organizing information in a structured manner. The **Answer Generation** stage leverages the LLM to generate answers using the retrieved context, producing accurate and contextually grounded responses. Finally, **Response Formatting** packages the answer along with source citations in a structured format, which is returned to the user through the API or frontend interface.

3.2.4 Technology Stack

The system is built using a modern technology stack that balances performance, maintainability, and ease of deployment. For document processing, the system employs Python 3.x as the primary programming language, along with PaddleOCR 3.2 [22], [23] for OCR capabilities, pdf2image for document conversion, and LibreOffice for handling various document formats. The embedding pipeline utilizes sentence-transformers library [5] with the all-MiniLM-L6-v2 model [6] for generating semantic embeddings, complemented by FlagEmbedding [12] for cross-encoder reranking functionality. Vector search is im-

plemented using FAISS [7], providing efficient similarity search over high-dimensional vectors. The backend infrastructure is built on FastAPI for RESTful API development, Pydantic for data validation, and Uvicorn as the ASGI server. The system supports multiple LLM providers including Google Gemini API for cloud-based generation, Ollama for local deployment, and Qwen [2] for both cloud and local inference scenarios. The frontend is constructed using React 19 for the user interface framework, Vite as the build tool, and TailwindCSS for styling. Finally, the entire system is containerized using Docker and Docker Compose, enabling consistent deployment across different environments.

Chapter 4

Experiments

In Chapter 4, it presents experimental evaluation of our RAG-based question-answering system for CSE course materials.

Throughout all experiments, we use a dataset collected from multiple sources including HCMUT LMS [20] and Coursewave [21]. The dataset encompasses multiple CSE courses, each containing syllabus documents and lecture materials (slides), enabling comprehensive evaluation across different course domains and document types. Table 4.1 provides an overview of the dataset composition.

Table 4.1: Dataset Statistics

Statistic	Value
Number of Courses	32
Total Documents	183
Syllabus Documents	153
Material Documents (Slides)	30
Total Text Chunks	15,013
Average Chunks per Course	469.2
Total Tokens (approximate)	6,140,317

To evaluate the effectiveness of different retrieval and query optimization strategies, we consider two baseline configurations that demonstrate the contribution

of retrieval components and query optimization:

RAG Baseline (FAISS + Reranker). This configuration implements the standard retrieval-augmented generation [3] pipeline with two-stage retrieval. The system first retrieves candidate chunks using FAISS [7] similarity search based on semantic embeddings, then applies cross-encoder reranking [11], [12] to refine the results by considering query-document interactions. The reranked chunks are subsequently passed to the LLM [13] for answer generation. This represents the main baseline for comparison, demonstrating the effectiveness of combining efficient vector search with precision-focused reranking.

RAG + Query Rewriter. This configuration extends the RAG baseline by incorporating query rewriting as an additional optimization step. Before retrieval, the original user query is rewritten by an LLM [13] to improve semantic matching with document content. The rewritten query is then used for both FAISS retrieval [7] and cross-encoder reranking [12], following the same two-stage retrieval pipeline as the baseline. This configuration tests whether query optimization through LLM-based rewriting improves retrieval quality and subsequent answer generation compared to the standard RAG baseline, particularly for ambiguous or conversational queries.

4.1 Answer Generation Quality

Answer generation quality is evaluated using semantic similarity metrics that do not require ground truth reference answers, making them suitable for datasets extracted from raw course materials. Two primary metrics are employed:

Query-Answer Semantic Similarity quantifies the semantic alignment between the original user query and the generated answer by computing the cosine similarity between their respective embeddings. The embeddings are generated using the same sentence-transformer model employed for document indexing, ensuring consistent semantic representation across the retrieval and evaluation pipeline. This metric serves as a proxy for answer relevance when reference

answers are unavailable.

Answer Faithfulness (also referred to as Answer-Chunk Semantic Similarity) measures the extent to which generated answers are grounded in the retrieved context. It is computed as the average cosine similarity between the answer embedding and the embeddings of the top- k retrieved chunks. This metric is critical for detecting hallucination and ensuring that answers are derived from the retrieved course materials rather than the language model’s parametric knowledge.

Both metrics are computed for each configuration (baseline and query rewriting variant) to enable comparative analysis of answer quality improvements.

4.2 Retrieval Performance

Retrieval performance is assessed using metrics that evaluate the relevance of retrieved chunks without requiring ground truth relevance labels:

Query-Chunk Semantic Similarity measures the average cosine similarity between the original query embedding and the embeddings of the top- k retrieved chunks. This metric quantifies how well the retrieved chunks match the query intent, using the same embedding model as the indexing process to ensure semantic consistency.

Top Chunk Score (Reranker Confidence) captures the reranker’s confidence score for the highest-ranked chunk. This metric provides insight into the reranker’s ability to identify the most relevant chunk at the top position, which is particularly important for answer generation quality, even when average similarity across all retrieved chunks may vary.

These metrics are computed for both the baseline configuration (FAISS + Reranker) and the query rewriting variant (FAISS + Query Rewriter + Reranker) to quantify the impact of query rewriting on retrieval quality.

4.3 Impact of Query Rewriting

The effectiveness of query rewriting is evaluated across three complementary dimensions to provide a comprehensive assessment. First, **Answer Quality Improvement** is measured as the change in Query-Answer Semantic Similarity and Answer Faithfulness metrics when using rewritten queries, indicating whether query rewriting produces more relevant and faithful answers compared to the baseline. Second, **Retrieval Quality Improvement** is quantified as the change in Query-Chunk Semantic Similarity and Top Chunk Score metrics, measuring whether rewritten queries retrieve more semantically relevant chunks. Third, **Query Type Analysis** provides a performance breakdown by query type (short ambiguous queries, course information queries, specific queries, and full questions) to identify query characteristics where rewriting provides the most benefit. This multi-dimensional analysis enables a comprehensive understanding of how query optimization affects both retrieval relevance and answer generation quality, with particular emphasis on answer faithfulness as a critical metric for educational RAG systems.

4.4 System Performance

System performance is evaluated in terms of computational efficiency and scalability:

Average query latency is measured in seconds, capturing the end-to-end time from query submission to answer generation. **Throughput** is quantified as queries processed per second, indicating the system’s capacity under the evaluated configurations.

A **latency breakdown** is performed across the major pipeline stages: query rewriting (when enabled), retrieval, reranking, and answer generation. This breakdown identifies performance bottlenecks and quantifies the computational overhead introduced by query rewriting.

Additionally, a **performance breakdown by query type** is conducted to understand how query rewriting affects different query categories, revealing whether certain query types experience disproportionate latency changes.

Finally, a **quality-latency trade-off** analysis is performed to assess whether any performance costs (or benefits) introduced by query rewriting are justified by corresponding improvements in answer quality and retrieval relevance.

4.5 Qualitative Analysis

Qualitative evaluation is conducted through case studies examining representative queries from each query type category (short ambiguous queries, course information queries, and specific queries). For each case study, **query transformation analysis** examines how query rewriting transforms the original query, including identification of expansion patterns, successful disambiguation, and instances of over-expansion that may dilute query intent. **Retrieved chunks comparison** analyzes the chunks retrieved before and after query rewriting, assessing how rewriting affects retrieval relevance and the semantic alignment between queries and retrieved content. **Answer quality comparison** compares generated answers and their corresponding semantic similarity scores (Query-Answer Similarity and Answer Faithfulness) before and after rewriting, evaluating whether rewritten queries produce more relevant and accurate answers. Finally, **success and failure pattern identification** systematically identifies query types and characteristics that benefit most from rewriting versus those that do not, to understand the conditions under which query rewriting is most effective. Best and worst case examples are selected based on the magnitude of similarity change to illustrate the relative strengths and limitations of the query rewriting approach. These examples provide insights into the mechanisms underlying query rewriting’s impact on retrieval and answer generation, complementing the quantitative metrics with qualitative understanding.

Chapter 5

Results

In Chapter 5, it presents experimental results comparing different RAG pipeline configurations.

5.1 Answer Generation Quality

Table 5.1 measures how well the system generates answers that are relevant to user queries and grounded in retrieved context. Two key metrics are analyzed: Query-Answer Semantic Similarity and Answer Faithfulness.

Table 5.1: Answer Generation Quality on CSE Course Materials

Metric	Baseline	Query Rewriter
Query-Answer Similarity	0.4977	0.4723
Answer Faithfulness	0.3646	0.4414

Query rewriting improves Answer Faithfulness by +21.1% while slightly decreasing Query-Answer Similarity (-5.1%). This trade-off prioritizes answer accuracy over direct query matching, reducing hallucination risk and ensuring answers are better grounded in retrieved context.

5.2 Retrieval Performance

Table 5.2 presents retrieval performance metrics for the two RAG configurations. These metrics measure the quality of retrieved chunks before answer generation.

Table 5.2: Retrieval Performance Metrics

Configuration	Query-Chunk Similarity	Top Chunk Score
RAG Baseline (FAISS + Reranker)	0.2453	-3.0721
RAG + Query Rewriter	0.1770	-0.5875

While Query-Chunk Similarity decreases (-27.8%), Top Chunk Score improves significantly (+80.9%). Query rewriting helps the reranker identify a highly relevant chunk at the top position, which is critical for answer generation. This improvement contributes to the +21.1% gain in Answer Faithfulness.

5.3 Impact of Query Rewriting

Table 5.3 compares the RAG baseline against the query rewriting variant.

Table 5.3: Impact of Query Rewriting on Answer Quality

Configuration	Query-Answer Sim	Answer Faithfulness	Top Chunk Score
RAG Baseline (FAISS + Reranker)	0.4977	0.3646	-3.0721
RAG + Query Rewriter	0.4723	0.4414	-0.5875
Improvement	-0.0254 (-5.1%)	+0.0768 (+21.1%)	+2.4846 (+80.9%)

Query rewriting improves Answer Faithfulness (+21.1%) and Top Chunk Score (+80.9%) with a minor decrease in Query-Answer Similarity (-5.1%). Rewritten queries better align with document embeddings, particularly benefiting ambiguous queries.

5.3.1 Impact by Query Type

Table 5.4 shows query rewriting impact by query type.

Specific queries benefit most (+6.5%), while short ambiguous and course info queries show decreases due to over-expansion. Full questions show marginal

Table 5.4: Query Rewriting Impact by Query Type

Query Type	Baseline	+ Rewriter	Δ Change
Specific	0.5797	0.6171	+0.0375 (+6.5%)
Full Question	0.7476	0.7661	+0.0185 (+2.5%)
Short Ambiguous	0.4494	0.4126	-0.0368 (-8.2%)
Course Info	0.4837	0.4530	-0.0307 (-6.3%)

improvement as they are already well-formed.

5.4 System Performance

Table 5.5 presents latency and throughput measurements for the two baseline configurations, highlighting the trade-offs between quality and performance.

Table 5.5: System Performance Benchmarks

Configuration	Avg Latency (s)	Throughput (qps)
RAG Baseline (FAISS + Reranker)	80.97	0.0124
RAG + Query Rewriter	72.49	0.0138
Improvement	-8.47 (-10.5%)	+0.0014 (+11.7%)

Query rewriting improves performance: latency decreases by 10.5% and throughput increases by 11.7%. Rewritten queries enable more efficient retrieval and faster reranker convergence. The improvement is most significant for short ambiguous queries (-30.6% latency).

5.4.1 Performance Breakdown by Query Type

Table 5.6 shows latency breakdown by query type, revealing where query rewriting provides the most performance benefit.

Table 5.6: Latency by Query Type

Query Type	Baseline (s)	+ Rewriter (s)	Improvement
Short Ambiguous	103.41	71.81	-30.6%
Course Info	65.94	75.46	+14.4%
Specific	39.16	79.28	+102.5%
Full Question	42.25	62.13	+47.1%

Short ambiguous queries show the largest latency improvement (-30.6%), while specific queries experience increased latency (+102.5%), indicating rewriting is unnecessary for well-formed queries.

5.5 Qualitative Analysis

This section presents qualitative examples comparing different configurations on representative queries, illustrating when query rewriting is most effective.

Example 1: Successful Expansion (Short Ambiguous Query)

- **Original Query:** "prerequisites"
- **Rewritten Query:** "prerequisites for the Introduction to Computing course, including any required knowledge in computer programming languages such as Python or Java, familiarity with basic data structures like arrays and linked lists, understanding of fundamental algorithms such as sorting and searching, and proficiency in using a text editor or integrated development environment (IDE) for writing code. Additionally, what grading criteria will be used for assignments and exams in this course?"
- **Course:** Introduction_to_Computing
- **Similarity Change:** +0.0480 (improved)
- **Analysis:** Successful expansion adds course-specific context, improving retrieval relevance.

Example 2: Over-Expansion (Short Ambiguous Query)

- **Original Query:** "prerequisites"
- **Rewritten Query:** "prerequisites for the Database Systems course, including any required knowledge in database management systems, programming languages such as SQL, and understanding of relational databases."

Additionally, what are the specific prerequisites needed to successfully complete this course?"

- **Course:** Database_Systems
- **Similarity Change:** -0.0089 (decreased)
- **Analysis:** Over-expansion adds terms (SQL, relational databases) not present in materials, causing slight similarity decrease.

Example 3: Best Improvement Case (Course Info Query)

- **Original Query:** "grading policy"
- **Rewritten Query:** "What are the grading policies for the Operating Systems course, including details on how assignments, exams, and participation will contribute to my final grade? Additionally, please specify any late submission policies, make-up exam procedures, and criteria for extra credit opportunities."
- **Course:** Operating_Systems
- **Baseline Similarity:** 0.4747
- **With Rewriter:** 0.6143
- **Improvement:** +0.1396 (+29.4%)
- **Analysis:** Best case. Expansion transforms general query into comprehensive question matching document structure.

Example 4: Worst Case (Already Specific Query)

- **Original Query:** "midterm"
- **Course:** Discrete_Structures_for_Computing
- **Baseline Similarity:** 0.5262

- **With Rewriter:** 0.1774
- **Decrease:** -0.3488 (-66.3%)
- **Analysis:** Worst case. Already-specific query is over-expanded, changing intent. Materials may lack detailed midterm information.

Query rewriting is most effective for ambiguous queries (e.g., "prerequisites", "grading policy") when expansion adds relevant context without over-expanding. It is least effective for already-specific queries (e.g., "midterm") where expansion may change intent or materials lack requested information.

Summary: 7/20 queries (35%) show improved similarity. Best: +29.4% ("grading policy"). Worst: -66.3% ("midterm").

Despite mixed individual query results (35% improved, 65% decreased similarity), Answer Faithfulness improves by +21.1% across all queries, indicating rewriting consistently improves answer accuracy by retrieving better context.

Chapter 6

Conclusion

6.1 Summary

This project successfully developed a Retrieval-Augmented Generation (RAG) system for CSE course materials at HCMUT, addressing the critical need for efficient and accurate course information retrieval. The system integrates document processing, semantic search using FAISS, reranking with FlagEmbedding, and answer generation through LLM integration. Through comprehensive evaluation, we demonstrated that the proposed RAG architecture effectively retrieves relevant course information and generates answers grounded in source materials, with query rewriting techniques providing significant improvements in answer faithfulness and retrieval quality.

6.2 Achievement of Objectives

This section evaluates the extent to which the project achieved its stated objectives:

Rapid Information Retrieval - The system successfully enables rapid information access compared to manual search methods. Experimental results show that the baseline RAG configuration achieves an average query latency of 80.97 sec-

onds, with the query rewriting variant further reducing latency to 72.49 seconds (10.5% improvement). While query rewriting introduces additional processing overhead, the overall system throughput improved by 11.7%, demonstrating that the architecture supports efficient information retrieval. However, it should be noted that the use of Qwen2.5:3B for query rewriting introduced significant latency, as the model's inference time contributed substantially to the end-to-end processing time. This limitation highlights the importance of model selection for production deployments where latency is critical.

High Accuracy and Reliability - The system achieved significant improvements in answer faithfulness, which directly addresses the critical requirement of minimizing hallucination in educational contexts. Query rewriting improved Answer Faithfulness by 21.1% (from 0.3646 to 0.4414), indicating that generated answers are better grounded in retrieved course materials. The Top Chunk Score improvement of 80.9% demonstrates that the reranker can more effectively identify highly relevant chunks when using rewritten queries, contributing to more accurate answer generation. While Query-Answer Similarity decreased slightly (-5.1%), this trade-off prioritizes answer accuracy over direct query matching, which is appropriate for educational applications where factual correctness is paramount.

Specific CSE Domain Query Handling - The system effectively handles various query types relevant to course selection and academic planning. Experimental analysis across four query categories (short ambiguous queries, course information queries, specific queries, and full questions) demonstrates the system's capability to process diverse information needs. Specific queries showed the most improvement (+6.5%) with query rewriting, while course information queries benefited from the enhanced retrieval quality. The qualitative analysis revealed successful handling of queries about prerequisites, grading policies, and course content, addressing the specific use cases outlined in the objectives.

Robust and Maintainable Architecture - The system was designed with mod-

ularity in mind, featuring separate components for preprocessing, indexing, retrieval, reranking, and answer generation. This architecture enables independent updates and scaling of individual components. The evaluation framework itself demonstrates this modularity, allowing systematic assessment of each component's contribution to overall system performance. The infrastructure supports future enhancements, as evidenced by the planned migration to vector databases and incremental indexing capabilities outlined in future work.

6.3 Key Findings

The experimental results reveal several important insights:

First, query rewriting significantly improves answer faithfulness (+21.1%) and top chunk retrieval quality (+80.9%), demonstrating that expanded queries better align with document embeddings and enable the reranker to identify more relevant context. This improvement is particularly valuable for educational applications where answer accuracy is critical.

Second, the impact of query rewriting varies substantially by query type. Specific queries benefit most (+6.5%), while short ambiguous queries and course information queries show mixed results, with some experiencing decreases due to over-expansion. This finding suggests that query rewriting should be applied selectively based on query characteristics, rather than universally.

Third, system performance improvements (10.5% latency reduction, 11.7% throughput increase) indicate that rewritten queries enable more efficient retrieval and faster reranker convergence, despite the additional processing overhead from query rewriting itself. However, the latency breakdown by query type reveals significant variation, with short ambiguous queries showing the largest improvement (-30.6%) while specific queries experience increased latency (+102.5%). Finally, the qualitative analysis demonstrates that query rewriting is most effective for ambiguous queries (e.g., "prerequisites", "grading policy") when expansion adds relevant context without over-expanding, and least effective for

already-specific queries where expansion may change intent or introduce irrelevant terms not present in the materials.

6.4 Future Directions

Future work will focus on two main directions: query processing improvements and system infrastructure enhancements.

For query processing, we plan to integrate Named Entity Recognition (NER) [24] or LLM-based entity extraction to automatically identify course names from user queries, enabling more precise course-specific retrieval without requiring explicit course selection. Additionally, we will explore advanced query rewriting techniques using LLMs to improve retrieval quality by reformulating ambiguous or incomplete queries.

For system infrastructure, we intend to migrate from FAISS to a vector database such as Qdrant [8] to support persistent storage, distributed deployment, and improved scalability for production environments. We also plan to implement incremental indexing capabilities to support dynamic updates to the knowledge base without requiring full re-indexing.

References

- [1] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models”, 2023.
- [2] Qwen et al., *Qwen2.5 technical report*, 2025. arXiv: [2412.15115 \[cs.CL\]](#).
- [3] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks”, in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.
- [5] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks”, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [6] W. Wang et al., “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers”, *arXiv preprint arXiv:2002.10957*, 2020.

- [7] M. Douze et al., *The faiss library*, 2025. arXiv: 2401.08281 [cs.LG].
- [8] Qdrant Team, *Qdrant: Vector database and similarity search engine*, <https://qdrant.tech/>, Accessed: 2024, 2024.
- [9] J. Wang et al., “Milvus: A purpose-built vector data management system”, in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, ACM, 2021, pp. 2614–2627.
- [10] Weaviate Team, *Weaviate: Open-source vector database*, <https://weaviate.io/>, Accessed: 2024, 2024.
- [11] R. Nogueira and K. Cho, “Passage re-ranking with bert”, in *arXiv preprint arXiv:1901.04085*, 2019.
- [12] Beijing Academy of Artificial Intelligence, *Bge reranker: Baai general embedding reranker*, <https://github.com/FlagOpen/FlagEmbedding>, Accessed: 2024, 2024.
- [13] J. Yang et al., “Harnessing the power of llms in practice: A survey on chatgpt and beyond”, 2023.
- [14] Y. Liu et al., “Roberta: A robustly optimized bert pretraining approach”, 2019.
- [15] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Al-bert: A lite bert for self-supervised learning of language representations”, 2019.
- [16] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pre-training text encoders as discriminators rather than generators”, in *International Conference on Learning Representations*, 2020.
- [17] H. Liu et al., “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.
- [18] H. Touvron et al., “Llama: Open and efficient foundation language models”, 2023.

- [19] L. Ouyang et al., “Training language models to follow instructions with human feedback”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.
- [20] HCMUT, *Bk-lms: Learning and teaching assistance system*, <https://lms.hcmut.edu.vn/>, Accessed: 2025, 2025.
- [21] HCMUT, *Hcmut courseware: All in one course materials repository*, <https://tinyurl.com/hcmut-courseware>, Accessed: 2025, 2025.
- [22] Y. Du et al., *Pp-ocr: A practical ultra lightweight ocr system*, 2020. arXiv: 2009.09941 [cs.CV].
- [23] PaddlePaddle, *Paddleocr: Ultra-lightweight ocr system*, <https://github.com/PaddlePaddle/PaddleOCR>, Accessed: 2025, 2024.
- [24] M. Marcinczuk, *Transformer-based named entity recognition with combined data representation*, 2024. arXiv: 2406.17474 [cs.LG].
- [25] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries”, in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81.

Appendix A

Source Code and Dataset Availability

1. Project Source Code

- **Project Name:** CSE Course RAG (Retrieval-Augmented Generation)
- **Platform:** GitHub
- **URL:** github.com/HatakekkSheeshh/CSE_course_RAG
- **Description:** Contains the core implementation of the RAG pipeline, including data ingestion, vector database configuration, and the inference logic.

2. Research Dataset

- **Dataset Name:** CSE Course RAG Dataset
- **Platform:** Hugging Face
- **URL:** huggingface.co/datasets/hatakekksheeshh/CSE_course_RAG
- **Description:** Consists of the curated academic course data, pre-processed text chunks, and evaluation sets used for testing the RAG system.